

```
"""
```

doi_resolver.py – GRUS Audit Pipeline Stage 3

Specification: GRUS-STD-001-v1.0 Section 3 Stage 3; Clause 6

Resolves the cited DOI, verifies it is publicly accessible, and confirms its timestamp is earlier than the audit date.

Published by: Green Recursive Utility Service LLC

```
"""
```

```
import urllib.request, urllib.error, json
from datetime import datetime, timezone
from dataclasses import dataclass
```

```
@dataclass
```

```
class DOIResult:
```

```
    passed: bool
    doi: str = ""
    resolved_url: str = ""
    deposit_timestamp: str = ""
    failure_reason: str = ""
```

```
def resolve_doi(doi: str, audit_date_iso: str) -> DOIResult:
```

```
    if not doi or not doi.startswith("10."):
        return DOIResult(passed=False, doi=doi,
                          failure_reason=f"Invalid DOI format: {doi}")
```

```
    url = f"https://doi.org/api/handles/{doi}"
```

```
    try:
```

```
        req = urllib.request.Request(url, headers={"Accept": "application/json"})
        with urllib.request.urlopen(req, timeout=30) as response:
            data = json.loads(response.read().decode())
```

```
    except urllib.error.HTTPError as e:
```

```
        return DOIResult(passed=False, doi=doi,
                          failure_reason=f"DOI resolution failed: HTTP {e.code}")
```

```
    except Exception as e:
```

```
        return DOIResult(passed=False, doi=doi,
                          failure_reason=f"DOI resolution error: {e}")
```

```
    resolved = ""
```

```
    for v in data.get("values", []):
```

```
        if v.get("type") == "URL":
            resolved = v.get("data", {}).get("value", "")
            break
```

```
    if not resolved:
```

```
        return DOIResult(passed=False, doi=doi,
                          failure_reason="DOI resolves but no URL field returned")
```

```
    deposit_ts = data.get("values", [{}])[0].get("timestamp", "")
```

```
    try:
```

```
        if deposit_ts:
            deposit_dt = datetime.fromisoformat(deposit_ts.replace("Z", "+00:00"))
            audit_dt = datetime.fromisoformat(audit_date_iso).replace(tzinfo=timezone.utc)
            if deposit_dt > audit_dt:
```

```

        return DOIResult(passed=False, doi=doi, resolved_url=resolved,
                           deposit_timestamp=deposit_ts,
                           failure_reason="DOI timestamp is later than audit date")
except Exception:
    pass
return DOIResult(passed=True, doi=doi, resolved_url=resolved,
                  deposit_timestamp=deposit_ts)

if __name__ == "__main__":
    import sys
    if len(sys.argv) < 2:
        print("Usage: python doi_resolver.py <doi> [audit_date_iso]"); sys.exit(0)
    doi = sys.argv[1]
    audit_date = sys.argv[2] if len(sys.argv) > 2 else datetime.utcnow().isoformat()
    r = resolve_doi(doi, audit_date)
    print(f"[{'PASS' if r.passed else 'FAIL'}] {r.doi}")
    if r.passed:
        print(f"  Resolved: {r.resolved_url}")
        print(f"  Deposit:   {r.deposit_timestamp}")
    else:
        print(f"  Reason: {r.failure_reason}")
        sys.exit(1)

```